



Pontificia Universidad Católica de Chile

Instituto de Ingeniería Matemática y Computacional

Magíster en Inteligencia Artificial

Tarea 2

Fundamentos Matemáticos para la
Inteligencia Artificial

IMT3850 · 2025-1

Alumno: Armando A.V.

Profesor: Manuel A. Sánchez

Fecha: Mayo 2026

Índice

1. Pregunta 1: Variables Aleatorias (5 puntos)	2
1.1. Solución	2
1.2. Verificación Numérica (Monte Carlo)	2
2. Pregunta 2: Algoritmo Random Quicksort (15 puntos)	2
2.1. Implementación	2
2.2. Experimentos	3
2.3. Resultados	3
2.4. Análisis	3
3. Pregunta 3: Convexidad (10 puntos)	3
3.1. Demostración ($\Rightarrow$)	4
3.2. Demostración ($\Leftarrow$)	4
4. Pregunta 4: Descenso de Gradiente (15 puntos)	4
4.1. Formulación	4
4.2. Implementación	5
4.3. Resultados	6
4.4. Análisis	6
5. Pregunta 5: Garantías Estadísticas y TLC (15 puntos)	6
5.1. a) Cota de Chebyshev	6
5.2. b) Teorema del Límite Central	7
5.3. Comparación	7
5.4. c-d) Simulación	7
6. Pregunta 6: Algoritmo de Mediana Aleatoria (10 pts — Bonus)	7
6.1. Algoritmo	7
6.2. Verificación	8
7. Conclusiones	8

1. Pregunta 1: Variables Aleatorias (5 puntos)

Enunciado: Sean x, y variables aleatorias independientes con $E[x] = E[y] = 0$ y $\text{Var}(x) = \text{Var}(y) = 1$. Determine la media μ_Z y la matriz de covarianza de $Z = (x, y, ax + by)$ con $a, b \in \mathbb{R}$.

1.1. Solución

Vector de medias:

$$\mu_Z = \begin{pmatrix} E[x] \\ E[y] \\ E[ax + by] \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

Matriz de covarianza: $\Sigma_Z = E[ZZ^\top]$

Usando independencia ($\text{Cov}(x, y) = 0$) y $\text{Var}(x) = \text{Var}(y) = 1$:

- $\text{Cov}(x, ax + by) = a\text{Var}(x) + b\text{Cov}(x, y) = a$
- $\text{Cov}(y, ax + by) = a\text{Cov}(y, x) + b\text{Var}(y) = b$
- $\text{Var}(ax + by) = a^2\text{Var}(x) + b^2\text{Var}(y) + 2ab\text{Cov}(x, y) = a^2 + b^2$

$$\Sigma_Z = \begin{pmatrix} 1 & 0 & a \\ 0 & 1 & b \\ a & b & a^2 + b^2 \end{pmatrix}$$

1.2. Verificación Numérica (Monte Carlo)

```

1 a, b = 2, 3
2 n_samples = 1_000_000
3 x = np.random.randn(n_samples)
4 y = np.random.randn(n_samples)
5 Z = np.column_stack([x, y, a*x + b*y])
6
7 print(f'mu_Z(empirico)={Z.mean(axis=0).round(4)}')
8 print(f'Sigma_Z(empirico)=\n{np.cov(Z.T).round(4)}')
9 # Resultado: mu ~ [0,0,0], Sigma ~ [[1,0,2],[0,1,3],[2,3,13]]

```

Listing 1: Verificación de Q1 por simulación

2. Pregunta 2: Algoritmo Random Quicksort (15 puntos)

2.1. Implementación

Se implementó el quicksort aleatorio con conteo de comparaciones. En cada llamada recursiva, se elige un pivote uniformemente al azar y se cuentan las comparaciones realizadas (una por cada elemento distinto al pivote).

```

1 def random_quicksort(S):
2     """Quicksort con pivot aleatorio. Retorna (lista_ordenada,
3         comparaciones)."""
4     if len(S) <= 1:
5         return list(S), 0
6     pivot = S[np.random.randint(len(S))]
7     S_minus = [x for x in S if x < pivot]
8     S_plus = [x for x in S if x > pivot]

```

```

8   S_eq      = [x for x in S if x == pivot]
9   comparisons = len(S) - len(S_eq)
10  left_sorted, left_comp = random_quicksort(S_minus)
11  right_sorted, right_comp = random_quicksort(S_plus)
12  return left_sorted + S_eq + right_sorted, comparisons + left_comp +
    right_comp

```

Listing 2: Quicksort aleatorio con conteo de comparaciones

2.2. Experimentos

```

1  n_values = list(range(100, 5001, 100))
2  n_trials = 30
3  avg_comparisons = []
4
5  for n in n_values:
6      comps = []
7      for _ in range(n_trials):
8          arr = list(np.random.permutation(n))
9          _, c = random_quicksort(arr)
10         comps.append(c)
11         avg_comparisons.append(np.mean(comps))
12
13 # Cotas teoricas
14 upper_bound = 2 * n_arr * np.log(n_arr) # 2n*ln(n)
15 lower_ref    = 2 * n_arr                # 2n

```

Listing 3: Experimentos para $n = 100, \dots, 5000$

2.3. Resultados

Se ejecutó el algoritmo para $n = 100, 200, \dots, 5000$ con 30 repeticiones por tamaño.

*[Gráfico generado en el notebook Jupyter]
Comparaciones promedio vs. $2n \ln(n)$ y $2n$*

Figura 1: Comparaciones del quicksort aleatorio vs. cotas teóricas.

2.4. Análisis

1. El número promedio de comparaciones crece como $O(n \log n)$, consistente con $E[C(n)] \leq 2n \ln(n)$.
2. El promedio empírico se mantiene por debajo de $2n \ln(n)$ para todos los tamaños.
3. El ratio empírico/ $2n \ln(n) \approx 0,7$, confirmando la constante teórica $\approx 1,39 \ln(n) = 2 \ln(n) / \ln(4)$.

3. Pregunta 3: Convexidad (10 puntos)

Enunciado: Sea $f : \mathbb{R}^d \rightarrow \mathbb{R}$ diferenciable. Demostrar:

$$f \text{ convexa} \iff f(y) \geq f(x) + \nabla f(x)^\top (y - x) \quad \forall x, y$$

3.1. Demostración ($\Rightarrow$)

Hipótesis: f convexa, i.e., $\forall x, y$ y $\theta \in [0, 1]$:

$$f(x + \theta(y - x)) \leq (1 - \theta)f(x) + \theta f(y)$$

Reordenando:

$$\frac{f(x + \theta(y - x)) - f(x)}{\theta} \leq f(y) - f(x)$$

Tomando $\theta \rightarrow 0^+$ (derivada direccional):

$$\nabla f(x)^\top (y - x) \leq f(y) - f(x)$$

Por lo tanto: $f(y) \geq f(x) + \nabla f(x)^\top (y - x)$. ✓

3.2. Demostración ($\Leftarrow$)

Hipótesis: $f(y) \geq f(x) + \nabla f(x)^\top (y - x)$ (*)

Sea $z = \theta y + (1 - \theta)x$ para $\theta \in [0, 1]$. Aplicamos (*) desde z :

1. Con $x \leftarrow z$, $y \leftarrow y$:

$$f(y) \geq f(z) + \nabla f(z)^\top (y - z) \quad (\text{i})$$

2. Con $x \leftarrow z$, $y \leftarrow x$:

$$f(x) \geq f(z) + \nabla f(z)^\top (x - z) \quad (\text{ii})$$

Multiplicamos (i) por θ y (ii) por $(1 - \theta)$ y sumamos:

$$\theta f(y) + (1 - \theta)f(x) \geq f(z) + \nabla f(z)^\top \underbrace{[\theta(y - z) + (1 - \theta)(x - z)]}_{=\theta y + (1 - \theta)x - z = 0}$$

Por lo tanto:

$$\boxed{\theta f(y) + (1 - \theta)f(x) \geq f(\theta y + (1 - \theta)x)}$$

Esto demuestra que f es convexa. ■

4. Pregunta 4: Descenso de Gradiente (15 puntos)

4.1. Formulación

Minimizar $L(\theta) = \frac{1}{2N} \|X\theta - y\|^2$ con:

- Gradiente: $\nabla L(\theta) = \frac{1}{N} X^\top (X\theta - y)$
- Gradiente estocástico (muestra i): $g_i = x_i(x_i^\top \theta - y_i)$
- Hessiano: $H = \frac{1}{N} X^\top X$

Tasas de aprendizaje:

- γ_{clases} (GD): Búsqueda exacta de línea $\gamma = \frac{g^\top g}{g^\top H g}$
- γ_{clases} (SGD): $\gamma = \frac{1}{L_{\text{máx}}}$ donde $L_{\text{máx}} = \max_i \|x_i\|^2$
- $\gamma(t) = \log(t + 2)$: Tasa creciente (divergencia esperada)

4.2. Implementación

```

1 def gradient_descent(X, y, NITMAX, gamma_fn, theta0=None, max_loss=1e15)
  :
2     N, D = X.shape
3     H = X.T @ X / N
4     theta = theta0.copy() if theta0 is not None else np.zeros(D)
5     losses = []
6     for t in range(NITMAX):
7         r = X @ theta - y
8         loss = 0.5 / N * np.dot(r, r)
9         losses.append(loss)
10        if loss > max_loss or np.isnan(loss):
11            break
12        g = X.T @ r / N
13        if np.linalg.norm(g) < 1e-12:
14            break
15        lr = gamma_fn(g, H, t)
16        theta = theta - lr * g
17    r = X @ theta - y
18    losses.append(0.5 / N * np.dot(r, r))
19    return theta, losses

```

Listing 4: Descenso de gradiente determinista

```

1 def sgd(X, y, NITMAX, gamma_fn, theta0=None, max_loss=1e15):
2     N, D = X.shape
3     H = X.T @ X / N
4     theta = theta0.copy() if theta0 is not None else np.zeros(D)
5     losses = []
6     for t in range(NITMAX):
7         if t % 10 == 0:
8             r = X @ theta - y
9             loss = 0.5 / N * np.dot(r, r)
10            losses.append(loss)
11            if loss > max_loss or np.isnan(loss):
12                break
13            i = np.random.randint(N)
14            xi = X[i]
15            gi = xi * (xi @ theta - y[i])
16            lr = gamma_fn(gi, H, t)
17            theta = theta - lr * gi
18    r = X @ theta - y
19    losses.append(0.5 / N * np.dot(r, r))
20    return theta, losses

```

Listing 5: Descenso de gradiente estocástico (SGD)

```

1 def gamma_clases_gd(g, H, t):
2     """Exact_line_search:  $\gamma = \frac{g^T T g}{g^T T H g}$ """
3     gHg = g @ H @ g
4     if gHg < 1e-15:
5         return 0.0
6     return (g @ g) / gHg
7
8 def make_gamma_clases_sgd(X):
9     """ $\gamma = 1/L_{\max}$ , estable para gradientes individuales."""
10    L_max = np.max(np.sum(X**2, axis=1))

```

```

11     def gamma_fn(g, H, t):
12         return 1.0 / L_max
13     return gamma_fn
14
15 def gamma_log(g, H, t):
16     """gamma(t) = log(t+2) -- crece sin limite."""
17     return np.log(t + 2)

```

Listing 6: Tasas de aprendizaje

4.3. Resultados

Método	γ	Lineal	Cuadrático
GD	γ_{clases} (exact LS)	Converge (~ 5 iter)	Converge (~ 10 iter)
GD	$\log(t+2)$	Diverge	Diverge
SGD	$1/L_{\text{máx}}$	Converge (con varianza)	Converge (con varianza)
SGD	$\log(t+2)$	Diverge	Diverge

Cuadro 1: Resumen de convergencia por método y tasa de aprendizaje.

4.4. Análisis

- La búsqueda exacta de línea es óptima para GD en funciones cuadráticas.
- Para SGD, $\gamma = 1/\max_i \|x_i\|^2$ garantiza estabilidad cuando se usan gradientes de una sola muestra.
- $\gamma = \log(t+2)$ crece sin cota, violando $\gamma < 2/\lambda_{\text{máx}}(H)$, causando divergencia.

5. Pregunta 5: Garantías Estadísticas y TLC (15 puntos)

Clasificador con $p = P(\text{acierto}) = 0,85$, $\varepsilon = 0,01$, $\delta = 0,05$.

5.1. a) Cota de Chebyshev

$$P(|\bar{X}_n - p| \geq \varepsilon) \leq \frac{\sigma^2}{n\varepsilon^2} \leq \delta$$

Despejando: $n \geq \frac{\sigma^2}{\varepsilon^2 \delta}$
 Con $\sigma^2 = p(1-p) = 0,1275$:

$$n_{\text{Cheb}} \geq \frac{0,1275}{0,01^2 \cdot 0,05} = 25,500$$

```

1 p, eps, delta = 0.85, 0.01, 0.05
2 sigma2 = p * (1 - p) # 0.1275
3 n_cheb = int(np.ceil(sigma2 / (eps**2 * delta)))
4 # n_cheb = 25,500

```

Listing 7: Cálculo tamaño de muestra — Chebyshev

5.2. b) Teorema del Límite Central

$$P(|\bar{X}_n - p| \geq \varepsilon) \approx 2 \left(1 - \Phi \left(\frac{\varepsilon \sqrt{n}}{\sigma} \right) \right) \leq \delta$$

Necesitamos: $\frac{\varepsilon \sqrt{n}}{\sigma} \geq z_{1-\delta/2} = 1,96$

$$n_{\text{CLT}} \geq \left(\frac{z_{0,975}}{\varepsilon} \right)^2 \sigma^2 = \left(\frac{1,96}{0,01} \right)^2 \cdot 0,1275 \approx 4,899$$

```

1 from scipy import stats
2 z = stats.norm.ppf(1 - delta/2) # z_0.975 = 1.96
3 n_clt = int(np.ceil((z / eps)**2 * sigma2))
4 # n_clt = 4,898

```

Listing 8: Cálculo tamaño de muestra — CLT

5.3. Comparación

El CLT requiere $\sim 5\times$ menos muestras que Chebyshev (4,899 vs 25,500), ya que usa la distribución asintótica en lugar de una cota universal.

5.4. c-d) Simulación

```

1 p_real = 0.85
2 n_test = n_clt
3 n_simulations = 10000
4
5 sample_means = np.array([
6     np.random.binomial(1, p_real, n_test).mean()
7     for _ in range(n_simulations)
8 ])
9
10 outside = np.mean(np.abs(sample_means - p_real) >= eps)
11 print(f'P(|X_bar - p| >= eps) empirica = {outside:.4f}')
12 print(f'Garantia delta = {delta}')
13 # Resultado: P empirica ~ 0.05, confirmando la garantia

```

Listing 9: Simulación para verificar garantía CLT

Se simularon 10,000 repeticiones con $n = n_{\text{CLT}}$ y $p = 0,85$. La proporción empírica $P(|\bar{X} - p| \geq \varepsilon) \approx 0,05 \leq \delta$, confirmando la garantía del CLT.

6. Pregunta 6: Algoritmo de Mediana Aleatoria (10 pts — Bonus)

6.1. Algoritmo

Implementación basada en el algoritmo de clase:

1. Muestrear $\sqrt{n} \cdot \log(n)$ elementos de S .
2. Ordenar la muestra y usar cuantiles para acotar el rango $[d, u]$ donde está la mediana.
3. Filtrar S al rango $[d, u]$ y verificar que la mediana esté contenida.
4. Si el filtro es válido, ordenar el subconjunto reducido y extraer la mediana.

5. Repetir si falla (probabilidad exponencialmente pequeña).

```

1 def random_median(S):
2     S = np.array(S, dtype=float)
3     n = len(S)
4     if n <= 5:
5         return float(np.median(S))
6     target = n // 2 # indice de la mediana
7
8     for attempt in range(100):
9         # Muestrear sqrt(n)*log(n) elementos
10        sample_size = min(n, max(10, int(np.sqrt(n) * np.log(n))))
11        R = np.random.choice(S, size=sample_size, replace=False)
12        R_sorted = np.sort(R)
13
14        # Cuantiles que acotan la mediana
15        k = len(R_sorted)
16        pos = k // 2
17        margin = max(1, int(np.sqrt(k) * 2))
18        lo_idx = max(0, pos - margin)
19        hi_idx = min(k - 1, pos + margin)
20        d, u = R_sorted[lo_idx], R_sorted[hi_idx]
21
22        # Filtrar al rango [d, u]
23        C = S[(S >= d) & (S <= u)]
24        l_d = np.sum(S < d)
25
26        # Verificar validez y extraer mediana
27        if l_d <= target and l_d + len(C) > target and len(C) <= 4*
28            sample_size:
29            C_sorted = np.sort(C)
30            return float(C_sorted[target - int(l_d)])
31
32    return float(np.median(S)) # fallback

```

Listing 10: Algoritmo aleatorio de mediana

6.2. Verificación

El algoritmo fue verificado para $n = 101, 1001, 5001, 10001$ contra `np.median`, obteniendo resultados idénticos en todos los casos.

7. Conclusiones

1. **Q1:** La media y covarianza se derivan de la linealidad de la esperanza y propiedades de combinaciones lineales.
2. **Q2:** El quicksort aleatorio confirma $O(n \log n)$ comparaciones en promedio.
3. **Q3:** La equivalencia convexidad $\iff$ condición de primer orden se demuestra en ambas direcciones.
4. **Q4:** La elección de γ es crítica: búsqueda exacta (GD) y $1/L_{\text{máx}}$ (SGD) convergen; $\log(t)$ diverge.
5. **Q5:** El TLC es más eficiente que Chebyshev ($\sim 5\times$ menos muestras).
6. **Q6:** El algoritmo aleatorio de mediana funciona en $O(n)$ esperado.